



Pashov Audit Group

OLY Security Review

February 16th 2026 - March 8th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. About OLY	4
6. Executive Summary	4
7. Findings	6
High findings	9
[H-01] Missing <code>launchTime</code> guard in <code>syncInflation</code> allows inflation manipulation prelaunch	9
Medium findings	11
[M-01] Internal revert bypasses try catch in <code>_getTokenPriceInUsd</code>	11
[M-02] StakingVault whitelist status not updated during address change	13
[M-03] AfterSwap logic skips limit orders at starting boundary during price drop	14
[M-04] Incorrect OlympusXHook tax state restoration in <code>Farmkeeper.depositToFarm</code>	17
[M-05] Distributed reward tokens lost during zero staker periods in <code>triggerCyclePayout</code>	18
[M-06] Stale stack variable renders swap retry mechanism non functional	20
[M-07] EthPrice uses incorrect <code>oracleStaleThreshold</code> for Chainlink stablecoin feeds	22
Low findings	26
[L-01] Wrong claimable amount emitted in auction claim event	26
[L-02] OLY pool ID cannot be reconfigured	26
[L-03] Incorrect ETH amount emitted in native swap event	27
[L-04] Restrictive slippage cap on liquidity removal can prevent farm removals	28
[L-05] Swap event reports incorrect tax values when tax is disabled for <code>FarmKeeper</code>	30
[L-06] LiquidityAction event includes accrued fees in reported token amounts	31
[L-07] Dust wstETH reward amount can revert harvest	32
[L-08] GoodAccountingExecuted event excludes reward penalties from total	33
[L-09] StakeOwner mapping not cleared when stakes are removed	34
[L-10] Dust reward tokens trapped in staking vault due to payout pool zeroing	35
[L-11] Unhandled chainlink revert bypasses twap fallback	36
[L-12] <code>setLaunchTime</code> can be called multiple times leading to time state corruption	37
[L-13] <code>FarmKeeper_Removed</code> event misrepresents ETH/WETH pool data and omits revenue	38
[L-14] Hardcoded Lido withdrawal limits may disrupt harvest after upgrade	39
[L-15] Missing initialization of <code>launchTime</code> and <code>lastInflationDistributionDay</code>	39
[L-16] Missing validation for <code>PoolKey</code> allows unsupported pools to lock fund	40
[L-17] V4 WETH pools route through intermediary instead of wrapping directly	41
[L-18] Incomplete interfaces missing external functions from implementation	43
[L-19] <code>harvestFarm</code> event emits current fee instead of total distributed fees	44
[L-20] ClaimRewards skips distributeDailyInflation unlike all other entry points	44
[L-21] <code>MarketCapLib.getMarketCapTier</code> has wrong tier boundary comment	45
[L-22] Claim all buyAndBurnWithdrawals DoS when any request is unfinalized	46
[L-23] Bonus formula deviates from whitepaper giving users fewer shares	46
[L-24] <code>addLiquidityTokens</code> missing <code>nonReentrant</code> - ERC-777 balance inflation	48
[L-25] <code>addRewardToken</code> has no cap unbounded gas DoS on core operation	50



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About OLY

OLY is a comprehensive token launch and DeFi yield platform that combines token minting with automatic staking, Uniswap V3/V4 liquidity farming, and Lido ETH staking across multiple reward cycles. The system features a governance-enabled staking vault with 5 different payout schedules, treasury-managed liquidity positions, and automated revenue distribution mechanisms.

6. Executive Summary

A time-boxed security review of the **De-centraX/olympusx-contracts** repository was done by Pashov Audit Group, during which **Ch301**, **Hals**, **h2134**, **VictoryGod**, **X0sauce** engaged to review OLY. A total of **33** issues were uncovered.

Protocol Summary

Project Name	OLY
Protocol Type	Yield management
Timeline	February 16th 2026 - March 8th 2026

Review commit hash:

- [3892211c9b35b3dc8d408dafa2a7dfd2d74e1466](#)
(De-centraX/olympusx-contracts)

Fixes review commit hash:

- [bb21fcc49e2f444a07f89806dc7cd5fb94c611bd](#)
(De-centraX/olympusx-contracts)

Scope

DefaultValues.sol FarmKeeperLib.sol FarmKeeperUniLib.sol
FarmKeeperV3Lib.sol FarmKeeperV4Lib.sol FarmKeeperViewLib.sol Farms.sol
GracePeriodPenalties.sol MarketCapLib.sol OlympusXHookLib.sol
StakingCalculations.sol SwapActionsLib.sol Common.sol EthPrice.sol



FarmKeeper.sol

LaunchManager.sol

OlympusX.sol

OlympusXAuction.sol

OlympusXHook.sol

StakingVault.sol

SwapActions.sol

TaxRouter.sol

ValidatorVault.sol



7. Findings

Findings count

Severity	Amount
High	1
Medium	7
Low	25
Total findings	33

Summary of findings

ID	Title	Severity	Status
[H-01]	Missing <code>launchTime</code> guard in <code>syncInflation</code> allows inflation manipulation prelaunch	High	Resolved
[M-01]	Internal revert bypasses try catch in <code>_getTokenPriceInUsd</code>	Medium	Resolved
[M-02]	StakingVault whitelist status not updated during address change	Medium	Resolved
[M-03]	AfterSwap logic skips limit orders at starting boundary during price drop	Medium	Resolved
[M-04]	Incorrect OlympusXHook tax state restoration in <code>Farmkeeper.depositToFarm</code>	Medium	Resolved
[M-05]	Distributed reward tokens lost during zero staker periods in <code>triggerCyclePayout</code>	Medium	Resolved
[M-06]	Stale stack variable renders swap retry mechanism non functional	Medium	Resolved
[M-07]	EthPrice uses incorrect <code>oracleStaleThreshold</code> for Chainlink stablecoin feeds	Medium	Resolved
[L-01]	Wrong claimable amount emitted in auction claim event	Low	Resolved
[L-02]	OLY pool ID cannot be reconfigured	Low	Resolved
[L-03]	Incorrect ETH amount emitted in native swap event	Low	Resolved



ID	Title	Severity	Status
[L-04]	Restrictive slippage cap on liquidity removal can prevent farm removals	Low	Resolved
[L-05]	Swap event reports incorrect tax values when tax is disabled for <code>FarmKeeper</code>	Low	Resolved
[L-06]	LiquidityAction event includes accrued fees in reported token amounts	Low	Resolved
[L-07]	Dust wstETH reward amount can revert harvest	Low	Resolved
[L-08]	GoodAccountingExecuted event excludes reward penalties from total	Low	Resolved
[L-09]	StakeOwner mapping not cleared when stakes are removed	Low	Resolved
[L-10]	Dust reward tokens trapped in staking vault due to payout pool zeroing	Low	Resolved
[L-11]	Unhandled chainlink revert bypasses twap fallback	Low	Resolved
[L-12]	<code>setLaunchTime</code> can be called multiple times leading to time state corruption	Low	Resolved
[L-13]	<code>FarmKeeper_Removed</code> event misrepresents ETH/WETH pool data and omits revenue	Low	Resolved
[L-14]	Hardcoded Lido withdrawal limits may disrupt harvest after upgrade	Low	Resolved
[L-15]	Missing initialization of <code>launchTime</code> and <code>lastInflationDistributionDay</code>	Low	Resolved
[L-16]	Missing validation for <code>PoolKey</code> allows unsupported pools to lock fund	Low	Resolved
[L-17]	V4 WETH pools route through intermediary instead of wrapping directly	Low	Resolved
[L-18]	Incomplete interfaces missing external functions from implementation	Low	Resolved
[L-19]	<code>harvestFarm</code> event emits current fee instead of total distributed fees	Low	Resolved
[L-20]	ClaimRewards skips distributeDailyInflation unlike all other entry points	Low	Resolved



ID	Title	Severity	Status
[L-21]	<code>MarketCapLib.getMarketCapTier</code> has wrong tier boundary comment	Low	Resolved
[L-22]	Claim all buyAndBurnWithdrawals DoS when any request is unfinalized	Low	Resolved
[L-23]	Bonus formula deviates from whitepaper giving users fewer shares	Low	Resolved
[L-24]	<code>addLiquidityTokens</code> missing <code>nonReentrant</code> - ERC-777 balance inflation	Low	Resolved
[L-25]	<code>addRewardToken</code> has no cap unbounded gas DoS on core operation	Low	Resolved



High findings

[H-01] Missing `launchTime` guard in `syncInflation` allows inflation manipulation prelaunch

Severity

Impact: High

Likelihood: Medium

Description

`StakingVault.syncInflation()` is a permissionless public function with no guard against being called before `LaunchManager.startMintPhase()` sets `launchTime` :

```
function syncInflation() public {
    _updateCurrentDay();
    _updateShareRate();
    _distributeDailyInflation();
}
```

Before `startMintPhase()` is called, `launchTime` is equal to 0. Any external caller can exploit this window:

```
// _updateCurrentDay() with launchTime = 0:
uint40 newDay = (uint40(block.timestamp) - 0) / uint40(1 days) + 1;
// newDay ≈ 20000+ (days since Unix epoch, Jan 1 1970)
currentDay = 20000+;

// _distributeDailyInflation():
lastInflationDistributionDay = 20000+; permanently corrupted
```

When `startMintPhase()` is later called, it sets `launchTime = block.timestamp`, making `currentDay = 1`. But `lastInflationDistributionDay` is already `20,000+` :

```
// _distributeDailyInflation() on every future call:
if (_lastDistributionDay >= _currentDay) return;
// 20000+ >= 1, will return
```

Inflation distribution is permanently bricked. No `OLY` inflation will ever be minted to stakers for the entire lifetime of the protocol. Replacing the `StakingVault` via `setStakingVault()` is not a viable recovery path, as reported separately (the missing initialization in `setStakingVault()` leaves any replacement vault equally broken).



Affected Code

`StakingVault.receive()` is also an affected entry point — any ETH sent to the vault before launch triggers `_updateCurrentDay()` through the receive fallback, causing the same `currentDay` corruption. The `launchTime != 0` guard must also be applied to the receive path.

Recommendation

Add a `launchTime` initialization guard to `StakingVault.syncInflation()` :

```
function syncInflation() public {  
+   require(launchTime > 0, StakingVault__NotLaunched());  
    _updateCurrentDay();  
    _updateShareRate();  
    _distributeDailyInflation();  
}
```



Medium findings

[M-01] Internal revert bypasses try catch in `_getTokenPriceInUsd`

Severity

Impact: High

Likelihood: Low

Description

`MarketCapLib._getTokenPriceInUsd()` wraps `EthPrice.getLatestEthPriceUSD()` in a try/catch to gracefully handle oracle failures. However, `getTokenPriceInETH()` is called *inside* the try body as an internal function. If `getTokenPriceInETH()` reverts (e.g., `FullMath.mulDiv` division by zero at extreme negative ticks), the revert is NOT caught because try/catch only captures reverts from the external call target. This propagates the revert to all callers including swap tax calculations, DoS-ing all swaps.

```
// MarketCapLib.sol L87-99
try EthPrice(ethPriceFeed).getLatestEthPriceUSD() returns (uint256 ethPriceUSD) {
    // Internal call – reverts bypass the catch block
    uint256 tokenPriceInETH = getTokenPriceInETH(poolManager, poolKey, twapPeriod,
oldestObservationSecondsAgo);
    return ethPriceUSD / tokenPriceInETH;
} catch {
    return 0; // never reached if getTokenPriceInETH reverts
}
```

PoC

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {EthPrice} from "@core/EthPrice.sol";
import {MockV3Aggregator} from "@mocks/MockV3Aggregator.sol";

/// @title M01_TryCatchBypassTest
/// @notice Demonstrates that try/catch only guards external calls – internal reverts
///         inside the try body bypass the catch block entirely. Mirrors the pattern
///         in MarketCapLib._getTokenPriceInUsd (L87-99) where
///         `getTokenPriceInETH()` is an internal call inside a try/catch wrapper.
contract M01_TryCatchBypassTest is Test {
    EthPrice ethPrice;

    function setUp() public {
        MockV3Aggregator feed = new MockV3Aggregator(8, 3000e8);
        ethPrice = new EthPrice(
            address(this),
```



```
EthPrice.PriceConfig({
  chainlinkFeed: address(feed),
  oracleStaleThreshold: 3600,
  uniswapFallback: EthPrice.UniswapConfig({
    isUniV4: false,
    poolAddress: address(0),
    weth: address(0),
    poolData: bytes(""),
    twapPeriod: 0,
    usdTokenChainlinkFeed: address(0)
  })
})
);
}

/// @dev Simulates the internal revert path (getTokenPriceInETH failure)
function _internalRevert() internal pure returns (uint256) {
  revert("internal fail");
}

/// @dev Mirrors MarketCapLib._getTokenPriceInUsd:
/// try EthPrice(ethPriceFeed).getLatestEthPriceUSD() returns (uint256 ethPriceUSD) {
///   uint256 tokenPriceInETH = getTokenPriceInETH(...); // internal – bypasses catch
///   return ethPriceUSD / tokenPriceInETH;
/// } catch { return 0; }
function getPriceUnsafe() public view returns (uint256) {
  try ethPrice.getLatestEthPriceUSD() returns (uint256 ethPriceUSD) {
    uint256 tokenPriceInETH = _internalRevert(); // internal revert bypasses catch
    return ethPriceUSD / tokenPriceInETH;
  } catch {
    return 0; // Never reached when internal call reverts
  }
}

function test_M01_tryCatch_bypass() public {
  // The external call to ethPrice.getLatestEthPriceUSD() succeeds,
  // but the internal _internalRevert() reverts – catch block NOT reached
  vm.expectRevert("internal fail");
  this.getPriceUnsafe();
}
}
```

Recommendations

Wrap the entire body in a secondary try/catch or move `getTokenPriceInETH` to an external call target:

```
try this._getTokenPriceInUsdExternal(ethPriceFeed, poolManager, poolKey, twapPeriod,
oldestObservationSecondsAgo)
  returns (uint256 price) {
    return price;
} catch {
  return 0;
}
```



[M-02] StakingVault whitelist status not updated during address change

Severity

Impact: Medium

Likelihood: Medium

Description

In the `OlympusX` contract, the `initializeWhitelist` method adds the initial `stakingVault` address to the minting whitelist (`isWhitelisted`).

[OlympusX.sol#L88-L89](#):

```
function initializeWhitelist(address _launchManager, address _stakingVault) external
onlyOwner {
    require(!isWhitelistedInitialized, OlympusX__WhitelistAlreadyInitialized());
    _whitelist(_launchManager);
    @> _whitelist(_stakingVault);
    @> stakingVault = _stakingVault;
    isWhitelistedInitialized = true;
}
```

However, the `setStakingVault` method updates only the `stakingVault` storage variable without modifying the whitelist status of the old or new vault addresses.

[OlympusX.sol#L95-L98](#):

```
function setStakingVault(address _stakingVault) external onlyOwner {
    require(_stakingVault != address(0), OlympusX__ZeroAddress());
    stakingVault = _stakingVault;
}
```

Impact

1. New `stakingVault` cannot mint OLY: The new `stakingVault` is not added to `isWhitelisted`, causing mint calls to revert.
2. Old `stakingVault` retains minting privileges: The deprecated vault remains whitelisted, posing a security risk if the old address is compromised or misused.

Recommendations

Modify `setStakingVault` method to synchronize whitelist status:

```
function setStakingVault(address _stakingVault) external onlyOwner {
    require(_stakingVault != address(0), OlympusX__ZeroAddress());
    + isWhitelisted[stakingVault] = false; // Remove old vault
    stakingVault = _stakingVault;
}
```



```
stakingVault = _stakingVault;  
+   isWhitelisted[stakingVault] = true; // Add new vault  
}
```

[M-03] AfterSwap logic skips limit orders at starting boundary during price drop

Severity

Impact: Medium

Likelihood: Medium

Description

The `_afterSwap` hook is responsible for filling limit orders that are crossed during a swap. However, the logic inside the iteration loop in `_afterSwap` contains mathematically flawed conditional checks. Specifically, during a downward price movement (tick decreasing), the condition `if (tickLowerLast < lower + key.tickSpacing)` always evaluates to `true` when the loop processes the starting boundary tick (`lower == tickLowerLast`).

This triggers a `continue` statement, permanently skipping the execution of `_fillEpoch` for the limit order residing at the exact tick where the swap began. Consequently, valid orders that have been fully crossed and utilized by the Uniswap pool are abandoned by the hook's accounting system.

Consider a standard market scenario where the price naturally rests inside a limit order's tick bucket, and then a subsequent swap pushes the price downward, fully crossing the order alongside others.

1. **State:** `tickLowerLast` (Start) = 123600. `tickLower` (End) = 123000. `tickSpacing` = 200.
2. **Direction:** Swap is selling ETH to buy OLY (Price DOWN). Swap `params.zeroForOne` is `true`. The hook's limit order direction variable is inverted: `bool zeroForOne = false`.
3. **Crossed Ticks Calculation:** `_getCrossedTicks` calculates the range. The loop iterates `lower` upwards from the end tick to the start tick: `123200`, `123400`, `123600`.
4. **Loop Execution (Boundary Tick):** When `lower` reaches `123600` (the starting tick):

```
solidity // Hook variable `zeroForOne` is false, so it enters the `else`  
block: } else { // tickLowerLast (123600) < lower (123600) + 200 // 123600 <  
123800 => TRUE if (tickLowerLast < lower + key.tickSpacing) { continue; } }
```


Because `123600 < 123800` evaluates to `true`, the `continue` statement executes. The order at `123600` is completely skipped, despite the fact that the Uniswap V4 Pool has just physically consumed its liquidity.



(Note: This vulnerability is strictly asymmetric. The code for upward price movements—the `if` block—is a mathematical impossibility since `lower` is capped at `tickLower - tickSpacing`. Therefore, upward limit orders execute flawlessly, while downward boundary orders are consistently broken.)

Proof of Concept

The following test places two identical Limit Sell orders.

Add this test to `OlympusXHook.LimitOrder.t.sol`:

```
// Helper to match the Hook's internal tick flooring math
function _getTickLower(int24 tick, int24 tickSpacing) private pure returns (int24) {
    int24 compressed = tick / tickSpacing;
    if (tick < 0 && tick % tickSpacing != 0) compressed--;
    return compressed * tickSpacing;
}

function test_POC_BoundaryTickOrderSkipped() public {
    // int24 tickSpacing = poolKey.tickSpacing; // To avoid Stack too deep just use `200`
    directly

    // 1. Disable tax to bypass Oracle Cardinality issues during test swaps
    vm.prank(admin);
    olympusXHook.setTaxSetterWhitelist(address(this), true);
    olympusXHook.setTaxEnabled(false);

    // Find the current tick bucket dynamically
    int24 currentTick = _getCurrentTick(); // 123890
    int24 currentBucket = _getTickLower(currentTick, 200); // 123800
    console.log("this is the currentTick", currentTick);
    console.log("this is the Tick after `_getTickLower`", currentBucket);

    // 2. Place Limit Sell Order (Sell OLY for ETH)
    // We place it 2 spacings below the current price
    // Place the order at (123800 - 400)
    int24 targetTick = currentBucket - (200 * 2); // 123400
    int24 targetTickB = currentBucket - 200; // 123600
    bool zeroForOne = false; // Sell OLY -> Buy ETH

    uint256 tokenAmount = 20e18;
    uint128 liquidity = _calculateLiquidity(targetTick, zeroForOne, 5e18);
    uint128 liquidityB = _calculateLiquidity(targetTick, zeroForOne, 5e18);

    _mintTokens(trader1, tokenAmount);
    vm.startPrank(trader1);
    olympusX.approve(address(olympusXHook), type(uint256).max);
    olympusXHook.place(poolKey, targetTick, zeroForOne, liquidity);
    olympusXHook.place(poolKey, targetTickB, zeroForOne, liquidityB);
    vm.stopPrank();

    Epoch epoch = olympusXHook.getEpoch(poolKey, targetTick, zeroForOne);
    Epoch epochB = olympusXHook.getEpoch(poolKey, targetTickB, zeroForOne);
```



```
// We use a massive swap amount, but rely on sqrtPriceLimitX96 to stop the swap
exactly where we want it.
uint256 massiveSwapAmount = 1000000 ether;
vm.deal(address(this), massiveSwapAmount * 2);

// 3. Swap #1: "Rest" the price exactly inside our target order's bucket.
// This simulates normal trading activity that updates `tickLowerLast` in the hook to
our `targetTick`.
// We aim for the middle of the bucket (targetTick + half spacing)
uint160 limitPrice1 = TickMath.getSqrtPriceAtTick(targetTick + (200 / 2));
_executeSwap(true, -int256(massiveSwapAmount), limitPrice1);

int24 tickAfterSwap1 = _getCurrentTick();
int24 bucketAfterSwap1 = _getTickLower(tickAfterSwap1, 200);
assertEq(bucketAfterSwap1, targetTick,
"Swap 1 must set the pool tick into the target order's bucket");

// 4. Swap #2: The Dump
// Push the price down past the order completely.
// Because `tickLowerLast` is now equal to `targetTick`, the flawed `if` statement
will trigger.
uint160 limitPrice2 = TickMath.getSqrtPriceAtTick(targetTick - (200 * 2));
_executeSwap(true, -int256(massiveSwapAmount), limitPrice2);

int24 tickAfterSwap2 = _getCurrentTick();
assertTrue(tickAfterSwap2 < targetTick, "Swap 2 must cross the order entirely");

// 5. Verify the Bug
// The order at targetTick has been fully crossed. It SHOULD be marked as filled.
(,, bool filled,,) = olympusXHook.getEpochInfo(epoch);
(,, bool filledB,,) = olympusXHook.getEpochInfo(epochB);

console.log("Is `123400` Filled? ", filled);
console.log("Is `123600` Filled? ", filledB);

// EXPECTATION: filled == true
// REALITY: filled == false (Because the faulty `if` statement skipped `lower ==
tickLowerLast`)
assertFalse(filled, "BUG CONFIRMED: Order at the boundary tick was skipped and left
unfilled!");
assertTrue(filledB, "All Good!");
}
```

Recommendations

```
// OlympusXHook.sol

for (; lower <= upper; lower += key.tickSpacing) {

-     if (zeroForOne) {
-         if (tickLower < lower + key.tickSpacing) {
-             continue;
-         }
-     } else {
-         if (tickLowerLast < lower + key.tickSpacing) {
-             continue;
-         }
-     }
}
```




```
-      }  
-    }  
    _fillEpoch(key, lower, zeroForOne);  
  }
```

[M-04] Incorrect OlympusXHook tax state restoration in Farmkeeper.depositToFarm

Severity

Impact: Medium

Likelihood: Medium

Description

The `depositToFarm` function in `FarmKeeper` temporarily disables the swap tax in `OlympusXHook` to avoid tax implications during liquidity operations, but unconditionally re-enables it afterwards without checking the original state. This overwrites any intentional tax deactivation set by authorized addresses.

[FarmKeeper.sol#L409](#):

```
function depositToFarm(bytes32 id, uint48 deadline) external beforeDeadline(deadline)  
validFarmId(id) nonReentrant {  
    Farm storage farm = _farms.get(id);  
    bool isOlympusX = (olympusXHook != address(0) && farm.id == olympusXPoolId);  
  
    if (isOlympusX) {  
        OlympusXHook(olympusXHook).setTaxEnabled(false);  
    }  
    _doDepositToFarm(farm, deadline);  
    if (isOlympusX) {  
@>        OlympusXHook(olympusXHook).setTaxEnabled(true);  
    }  
}
```

When the protocol administrator or an authorized entity explicitly sets `taxEnabled` to `false` (e.g., for maintenance, emergency response, or specific operational periods), any subsequent call to `depositToFarm` will forcibly re-enable the tax. This creates unexpected behavior where the tax state toggles from `false` to `true` without authorization, potentially exposing users to unexpected fees or interfering with protocol governance decisions.



Vulnerability Path:

1. Admin calls `OlympusXHook.setTaxEnabled(false)` to pause taxes during system maintenance.
2. User calls `FarmKeeper.depositToFarm()` to add liquidity.
3. Function execution: sets tax to **false** → deposits → forces tax back to **true**.
4. Tax is unexpectedly re-enabled, violating the admin's maintenance configuration.

Recommendations

Cache the original `taxEnabled` state before modification and restore it after the operation, rather than assuming it should always revert to true.

[M-05] Distributed reward tokens lost during zero staker periods in `triggerCyclePayout`

Severity

Impact: Medium

Likelihood: Medium

Description

`StakingVault` initializes `DEAD_SHARES = 1000` permanently assigned to `address(0xdead)`, meaning `totalShares` is never zero. `_distributeDailyInflation()` correctly accounts for this with an early return guard:

```
function _distributeDailyInflation() internal {
    //...
    if (_totalShares == DEAD_SHARES) {
        lastInflationDistributionDay = _currentDay;
        return; // <-- skips inflation when no real stakers
    }
    //...
}
```

Additionally, dead shares silently absorb a fraction (`DEAD_SHARES / totalShares`) of every daily inflation distribution in `_distributeDailyInflation()` even when real stakers exist. This permanently inflates `totalInflationDistributed` against the `maxInflationPool` cap without benefiting real stakers and silently burns a small fraction of all inflation over the protocol's lifetime. The fix in `_distributeDailyInflation()` should use `effectiveShares = totalShares - DEAD_SHARES` as the denominator rather than `totalShares`.

However, `_triggerCyclePayout()` has no equivalent guard:



```
function _triggerCyclePayout(uint256 _cycleNo, uint256 _globalActiveShares, ...) internal {
    if (_currentContractDay < nextCyclePayoutDay[_cycleNo]) return;

    if (_globalActiveShares > 0) { // <-- passes when only DEAD_SHARES=1000 exist
        for (uint256 i = 0; i < tokensLength; i++) {
            uint256 tokenReward = tokenCyclePayoutPool[token][_cycleNo];
            if (tokenReward > 0) {
                uint256 increment = (tokenReward * PRECISION) / _globalActiveShares;
                cumulativeRewardPerShare[token][_cycleNo] += increment;
                tokenCyclePayoutPool[token][_cycleNo] = 0; // <-- rewards cleared
            }
        }
    }
}
```

```
function _addToPools(address _token, uint256 _amount) internal {
    tokenCyclePayoutPool[_token][CYCLE_8_DAYS] += amount8;
    tokenCyclePayoutPool[_token][CYCLE_28_DAYS] += amount28;
    // ...
    _triggerPayouts();
}
```

When a cycle trigger condition is met and `totalShares == DEAD_SHARES`, the entire `tokenCyclePayoutPool` balance is distributed into `cumulativeRewardPerShare` with only dead shares as the denominator. As a result, all distributed reward tokens are permanently lost during a period (when the time to payout for that cycle is reached, `_currentContractDay >= nextCyclePayoutDay[_cycleNo]`) where no real stakers exist.

This is not limited to the launch window. It can occur at any point during the protocol's lifetime whenever all real stakes are ended, leaving only dead shares.

Recommendation

Apply the same guard used in `_distributeDailyInflation()` to `_triggerCyclePayout()`, which preserves accumulated rewards in `tokenCyclePayoutPool` across cycles until real stakers return, at which point the full accumulated balance is distributed correctly:

```
function _triggerCyclePayout(
    uint256 _cycleNo,
    uint256 _globalActiveShares,
    uint256 _currentContractDay
) internal {
    //...
    if (_globalActiveShares <= DEAD_SHARES) return;
    //...
}
```



[M-06] Stale stack variable renders swap retry mechanism non functional

Severity

Impact: Medium

Likelihood: Medium

Description

Both `_swapV3Token()` and `_swapV4Currency()` initialize a stack variable `amountToSwap` from `params.amountIn` before the retry loop. The catch block correctly reduces `retryData.amountToSwap`, but the swap call always reads the stale stack variable. Every retry uses the original full amount, making all retries after the first guaranteed to fail identically.

```
// SwapActions.sol L161
uint256 amountToSwap = params.amountIn; // initialized ONCE

RetryData memory retryData = RetryData({amountToSwap: amountToSwap, ...});
while (retryData.amountToSwap > 0 && retryData.retryCount < maxRetries) {
    try SwapActionsLib.swapExactInV3(
        ...,
        amountToSwap, // STALE — always equals params.amountIn
        ...
    ) returns (uint256 amountOut) { ... }
    catch (bytes memory reason) {
        retryData.amountToSwap = _getNewSwapAmountForRetry(reason,
            retryData.amountToSwap, ...);
        // reduces struct field, but swap call never reads it
    }
}
```

The same pattern exists in `_swapV4Currency()` at L212 with `uint128 amountToSwap`.

PoC:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Base} from "@test/Base.t.sol";

/// @title M06_StaleRetryVariableTest
/// @notice Demonstrates that SwapActions._swapV3Token (L162) initializes
///         `uint256 amountToSwap = params.amountIn` as a stack variable, then
///         passes it to every retry swap call. Although `retryData.amountToSwap`
///         is correctly reduced by `swapAmountForRetryBPS` (75%), the stale stack
///         variable means every retry swaps the full original amount.
contract M06_StaleRetryVariableTest is Base {
    function test_M06_allRetriesUseSameAmount() public view {
        // Read the actual retry BPS from FarmKeeper (inherits SwapActions)
    }
}
```



```
uint256 RETRY_BPS = farmKeeper.swapAmountForRetryBPS(); // 75_00 = 7500
uint256 BPS_TOTAL = 10000;
assertEq(RETRY_BPS, 7500); // Confirms the real contract value

uint256 paramsAmountIn = 100 ether;
uint256 amountToSwap = paramsAmountIn; // stack variable – NEVER updated
uint256 retryDataAmount = amountToSwap;

uint256[10] memory swapCallAmounts;
for (uint8 i; i < 10; i++) {
    swapCallAmounts[i] = amountToSwap; // uses stale stack var
    retryDataAmount = retryDataAmount * RETRY_BPS / BPS_TOTAL; // correctly reduces
struct
}

// All swap calls used the same amount (the bug)
for (uint8 i; i < 10; i++) {
    assert(swapCallAmounts[i] == paramsAmountIn);
}
// But retryData was reduced
assert(retryDataAmount < paramsAmountIn);
}

function test_M06_fixedVersionReducesProgressively() public view {
    uint256 RETRY_BPS = farmKeeper.swapAmountForRetryBPS();
    uint256 BPS_TOTAL = 10000;

    uint256 paramsAmountIn = 100 ether;
    uint256 retryDataAmount = paramsAmountIn;

    uint256[10] memory swapCallAmounts;
    for (uint8 i; i < 10; i++) {
        swapCallAmounts[i] = retryDataAmount; // FIXED: uses struct field
        retryDataAmount = retryDataAmount * RETRY_BPS / BPS_TOTAL;
    }

    assert(swapCallAmounts[0] == 100 ether);
    assert(swapCallAmounts[1] == 75 ether);
    assert(swapCallAmounts[2] == 56.25 ether);
    // Each retry uses a smaller amount
    for (uint8 i = 1; i < 10; i++) {
        assert(swapCallAmounts[i] < swapCallAmounts[i - 1]);
    }
}
```

Recommendations

Replace `amountToSwap` with `retryData.amountToSwap` in the swap call:

```
- amountToSwap,
+ retryData.amountToSwap,
```

Or remove the separate stack variable entirely.



[M-07] EthPrice uses incorrect `oracleStaleThreshold` for Chainlink stablecoin feeds

Severity

Impact: Medium

Likelihood: Medium

Description

`EthPrice.sol` uses a single `oracleStaleThreshold` of 1 hour for both the primary ETH/USD feed and the secondary USD token feeds (USDT/USD, USDC/USD). These feeds have different Chainlink heartbeats:

Feed	Heartbeat	Deviation
ETH/USD	1 hour	0.5%
USDT/USD	24 hours	0.1%
USDC/USD	24 hours	0.1%

The 1-hour threshold is correct for ETH/USD but wrong for the stablecoin feeds. Since USDT/USD and USDC/USD only update every 24 hours (unless the deviation exceeds 0.1%), the contract falsely rejects 95.8% of their valid data as "stale."

Conversely, relaxing the single threshold to accommodate the 24-hour stablecoin heartbeat could make it too lenient for the ETH/USD feed (1-hour heartbeat), allowing stale ETH prices to be accepted. A per-feed threshold is the only correct solution.

In `_getPriceFromChainlink()`, the same `oracleStaleThreshold` is used regardless of which feed is being queried:

```
// EthPrice.sol L124
if (block.timestamp > updatedAt + oracleStaleThreshold || priceInt <= 0) {
    return 0;
}
```

This threshold is passed from `getLatestEthPriceUSD()` into both the primary check and `_getUniswapTWAP()`, where it is reused for the secondary feeds:

```
// EthPrice.sol L169-172
if (config.usdTokenChainlinkFeed != address(0)) {
    uint256 usdTokenPrice = _getPriceFromChainlink(config.usdTokenChainlinkFeed,
    oracleStaleThreshold);
}
```

When the secondary feed is older than 1 hour (normal for a 24-hour heartbeat), `_getPriceFromChainlink` returns 0, and the USD conversion is silently skipped. The ETH price is then returned in USDT/USD terms rather than USD terms.



During a stablecoin depeg (e.g., USDT at \$0.95), this causes a 5.26% price inflation. LaunchManager, BuyAndBurn, and all other consumers of `getLatestEthPriceUSD()` receive incorrect prices.

PoC

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {EthPrice} from "@core/EthPrice.sol";
import {MockV3Aggregator} from "@mocks/MockV3Aggregator.sol";
import {MockUniswapV3Pool} from "@mocks/MockUniswapV3Pool.sol";
import {MockWETH} from "@mocks/MockWETH.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockUSDT18H02 is ERC20 {
    constructor() ERC20("Tether USD", "USDT") {}
    function mint(address to, uint256 amount) external { _mint(to, amount); }
}

contract EthPriceWrongStaleThresholdTest is Test {
    EthPrice public ethPrice;
    MockV3Aggregator public primaryFeed;
    MockV3Aggregator public secondaryFeed;
    MockUniswapV3Pool public pool;
    MockWETH public weth;
    MockUSDT18H02 public usdt;

    uint256 constant ETH_USD_THRESHOLD = 1 hours;
    uint256 constant USDT_HEARTBEAT = 24 hours;
    uint32 constant TWAP_PERIOD = 900;

    function setUp() public {
        vm.warp(1_000_000);
        weth = new MockWETH();
        usdt = new MockUSDT18H02();

        address t0 = address(weth) < address(usdt) ? address(weth) : address(usdt);
        address t1 = address(weth) < address(usdt) ? address(usdt) : address(weth);
        pool = new MockUniswapV3Pool(t0, t1);

        bool wethIsToken0 = address(weth) < address(usdt);
        int24 targetTick = wethIsToken0 ? int24(80068) : int24(-80068);
        pool.setSlot0Data(0, targetTick, 0, 1);

        primaryFeed = new MockV3Aggregator(8, 3000e8);
        secondaryFeed = new MockV3Aggregator(8, 100_000_000);

        EthPrice.UniswapConfig memory uniConfig = EthPrice.UniswapConfig({
            poolAddress: address(pool),
            weth: address(weth),
            twapPeriod: TWAP_PERIOD,
            isUniV4: false,
            poolData: "",
            usdTokenChainlinkFeed: address(secondaryFeed)
        });
    }
};
```



```
EthPrice.PriceConfig memory config = EthPrice.PriceConfig({
    chainlinkFeed: address(primaryFeed),
    oracleStaleThreshold: ETH_USD_THRESHOLD,
    uniswapFallback: uniConfig
});

ethPrice = new EthPrice(address(this), config);
}

function test_H02_depeggedUsdtSkippedAt4Hours() public {
    // Make primary stale to force TWAP path
    vm.warp(block.timestamp + ETH_USD_THRESHOLD + 1);

    // Set USDT at $0.95 depeg, fresh
    secondaryFeed.updateAnswer(95_000_000);
    uint256 secondaryUpdateTime = block.timestamp;

    // Secondary fresh -> conversion applied
    uint256 priceCorrect = ethPrice.getLatestEthPriceUSD();

    // Advance 4 hours: secondary valid per 24hr heartbeat but > 1hr threshold
    vm.warp(secondaryUpdateTime + 4 hours);
    uint256 priceWrong = ethPrice.getLatestEthPriceUSD();

    // Price inflated ~5.26% because USD conversion was skipped
    assertGt(priceWrong, priceCorrect, "4hr-old valid USDT data rejected, price inflated");
    uint256 errorBps = ((priceWrong - priceCorrect) * 10_000) / priceCorrect;
    assertGt(errorBps, 400, "Error exceeds 4%");
}

function test_H02_validUsdtDataRejectedByWrongThreshold() public {
    primaryFeed.updateAnswer(3000e8);
    secondaryFeed.updateAnswer(95_000_000);
    uint256 T = block.timestamp;

    // Fresh secondary: conversion applied (correct price)
    vm.warp(T);
    primaryFeed.updateAnswer(3000e8);
    vm.warp(T + ETH_USD_THRESHOLD);
    secondaryFeed.updateAnswer(95_000_000);
    vm.warp(T + ETH_USD_THRESHOLD + 1);
    uint256 correctPrice = ethPrice.getLatestEthPriceUSD();

    // 4hr-old secondary: conversion skipped (wrong price)
    vm.warp(T + ETH_USD_THRESHOLD + 4 hours + 1);
    uint256 wrongPrice = ethPrice.getLatestEthPriceUSD();

    assertGt(wrongPrice, correctPrice, "Valid 4hr-old USDT data rejected");
}

function test_H02_thresholdMismatchCoverage() public pure {
    uint256 validWindow = 1 hours;
    uint256 heartbeat = 24 hours;
    uint256 brokenPercentage = ((heartbeat - validWindow) * 100) / heartbeat;
```




```
        assertGt(brokenPercentage, 90, "Bug is active >90% of the time");
    }
}
```

Recommendations

Add a separate staleness threshold for the USD token feeds:

```
struct UniswapConfig {
    // ... existing fields ...
    address usdTokenChainlinkFeed;
+   uint256 usdTokenStaleThreshold;
}

function _getUniswapTWAP(...) internal view returns (uint256 price) {
    // ...
    if (config.usdTokenChainlinkFeed != address(0)) {
-       uint256 usdTokenPrice = _getPriceFromChainlink(config.usdTokenChainlinkFeed,
oracleStaleThreshold);
+       uint256 usdTokenPrice = _getPriceFromChainlink(config.usdTokenChainlinkFeed,
config.usdTokenStaleThreshold);
    }
}
```

Set `usdTokenStaleThreshold = 86400` (24 hours) for USDT/USD and USDC/USD feeds.



Low findings

[L-01] Wrong claimable amount emitted in auction claim event

Description

When a bidder claims their `OLY` tokens from `OlympusXAuction::claim()`, the `UserClaimed` event emits `claimableAmount` as the amount received by the user. However, if the `OlympusXAuction` contract or the claiming user is taxable in `OLY::_update()`, a transfer tax is applied during the `safeTransfer` call. This means the actual amount of tokens received by the user is less than `claimableAmount`, making the emitted event inaccurate.

Off-chain systems and indexers that rely on the `UserClaimed` event to track claimed amounts will record incorrect values, since the emitted amount does not reflect the transfer tax deducted during the token transfer.

The recommendation is to emit the actual amount of tokens received by the user after the transfer tax has been applied, rather than the pre-tax `claimableAmount`. This can be achieved by comparing the user token balance before and after the transfer.

```
function claim(uint256 cycle) external override nonReentrant {
    if (cycle == currentCycle && block.timestamp < auctionEndTime) {
        revert OlympusXAuction__AuctionNotEnded();
    }

    uint256 claimableAmount = getClaimableAmount(msg.sender, cycle);
    if (claimableAmount == 0) revert OlympusXAuction__NothingToClaim();

    userDeposits[msg.sender][cycle] = 0;
    totalClaimedPerCycle[cycle] = totalClaimedPerCycle[cycle] + claimableAmount;
    totalReservedTokens -= claimableAmount;

    @> IERC20(address(olympusX)).safeTransfer(msg.sender, claimableAmount);

    @> emit UserClaimed(msg.sender, cycle, claimableAmount);
}
```

[L-02] OLY pool ID cannot be reconfigured

Description

The `olympusXPoolId` in `FarmKeeper` is set once during `enableOlympusXFarm()` and there is no mechanism to update it afterward.

```
function enableOlympusXFarm(EnableFarmParams calldata params, uint256 amount0, uint256
amount1, uint48 deadline)
    external
```



```
payable
beforeDeadline(deadline)
{
    require(msg.sender == launchManager, FarmKeeper_NotAuthorized());
    PoolId id = params.poolKey.toId();
    bytes32 idBytes = PoolId.unwrap(id);
    require(!_farms.contains(id), FarmKeeper_DuplicatedFarm());

    @>
        olympusXPoolId = idBytes;
        _addFarm(idBytes, params, params.poolKey, true, FarmType.V4);
        // ...
}
```

The `olympusXPoolId` is used in `depositToFarm()` to identify the OLY farm and temporarily disable the sell tax during deposits.

```
function depositToFarm(bytes32 id, uint48 deadline) external beforeDeadline(deadline)
validFarmId(id) nonReentrant {
    Farm storage farm = _farms.get(id);
    @>
        bool isOlympusX = (olympusXHook != address(0) && farm.id == olympusXPoolId);

    if (isOlympusX) {
        OlympusXHook(olympusXHook).setTaxEnabled(false);
    }
    _doDepositToFarm(farm, deadline);
    if (isOlympusX) {
        OlympusXHook(olympusXHook).setTaxEnabled(true);
    }
}
```

If the protocol needs to migrate the OLY <> ETH pool to a new V4 pool with a different hook, the `olympusXPoolId` cannot be updated. This means the new pool's farm will not be recognized as the OLY farm during deposits, and the sell tax will not be disabled for it. The old pool, if it remains as a farm, would continue to receive the tax exemption and also continue to receive ETH reward allocations.

The recommendation is to expose a setter function restricted to the owner that allows `olympusXPoolId` to be updated when the OLY pool needs to be migrated to a new configuration.

[L-03] Incorrect ETH amount emitted in native swap event

Description

In `FarmKeeperUniLib::processEthSwapResult()`, the `FarmKeeper_NativeSwapped` event emits `ethAmount`, which represents the full `farm.nativeBalance` before the swap. However, if the swap has an `unusedAmount` due to retry failures or partial fills, the actual ETH consumed is `ethAmount - swapResult.unusedAmount`.

```
function processEthSwapResult(
    FarmType farmType,
    Farm storage farm,
```



```
PoolData memory intermediaryPool,  
SwapActions.SwapExactInResult memory swapResult,  
mapping(address => bool) storage isWethToken,  
uint256 ethAmount  
) external {  
    if (farmType == FarmType.V3) {  
        FarmKeeperV3Lib.processEthSwapResult(farm, intermediaryPool, swapResult,  
isWethToken);  
    } else {  
        FarmKeeperV4Lib.processEthSwapResult(farm, intermediaryPool, swapResult);  
    }  
    @> emit FarmKeeper.FarmKeeper_NativeSwapped(farm.id, ethAmount, swapResult.amountOut);  
}
```

The `ethAmount` passed to the function is the original `farm.nativeBalance` from both `_swapV3EthForFarmToken()` and `_swapV4EthForFarmToken()`.

```
function _swapV4EthForFarmToken(Farm storage farm, PoolData memory intermediaryPool,  
uint48 deadline) private {  
    @> uint128 ethAmount = farm.nativeBalance > type(uint128).max ? type(uint128).max :  
uint128(farm.nativeBalance);  
    if (ethAmount == 0) return;  
    // ...  
    @> FarmKeeperUniLib.processEthSwapResult(FarmType.V4, farm, intermediaryPool, result,  
isWethToken, ethAmount);  
}
```

When `swapResult.unusedAmount > 0`, the event reports more ETH as swapped than was actually consumed. This can mislead off-chain monitoring systems and indexers that rely on this event for accurate accounting. The recommendation is to emit `ethAmount - swapResult.unusedAmount` instead of the full `ethAmount` in the `FarmKeeper_NativeSwapped` event.

[L-04] Restrictive slippage cap on liquidity removal can prevent farm removals

Description

When removing liquidity from a farm, both `FarmKeeperV3Lib::decreaseLiquidity()` and `FarmKeeperV4Lib::_prepareDecreaseLiquidityUnlockData()` enforce minimum output amounts based on the farm's `liquiditySlippage` setting. The `validateLiquiditySlippage()` function caps this value at a maximum of 3000 (30%), meaning the minimum amount returned must be at least 70% of the expected amount based on the TWAP price.

```
function validateLiquiditySlippage(uint16 slippage) public pure {  
    @> if (slippage < 1 || slippage > 3000) revert FarmKeeper_InvalidSlippage();  
}
```

In `FarmKeeperV3Lib`, the minimum amounts for decreasing liquidity are calculated as follows.



```
function _prepareDecreaseLiquidity(
    uint256 tokenId,
    uint128 liquidity,
    uint256 deadline,
    uint256 slippage,
    uint160 twaPrice,
    int24 spacing
) private pure returns (INonfungiblePositionManager.DecreaseLiquidityParams memory
decreaseLiquidityParams) {
    (uint256 minAmount0, uint256 minAmount1) = LiquidityAmounts.getAmountsForLiquidity(
        twaPrice,
        TickMath.getSqrtPriceAtTick(TickMath.minUsableTick(spacing)),
        TickMath.getSqrtPriceAtTick(TickMath.maxUsableTick(spacing)),
        liquidity
    );

    @> minAmount0 = (minAmount0 * (BPS - slippage)) / BPS;
    @> minAmount1 = (minAmount1 * (BPS - slippage)) / BPS;
    // ...
}
```

Similarly, in `FarmKeeperV4Lib` :

```
function _prepareDecreaseLiquidityUnlockData(
    Farm storage farm,
    PoolKey memory poolKey,
    uint256 positionId,
    uint128 liquidity
) internal returns (bytes memory unlockData) {
    // ...
    (uint256 minAmount0, uint256 minAmount1) = LiquidityAmounts.getAmountsForLiquidity(
        V4OracleLib.getTwapSqrtPriceX96(farm.poolKey, farm.twapPeriod),
        TickMath.getSqrtPriceAtTick(TickMath.minUsableTick(poolKey.tickSpacing)),
        TickMath.getSqrtPriceAtTick(TickMath.maxUsableTick(poolKey.tickSpacing)),
        liquidity
    );

    @> minAmount0 = (minAmount0 * (BPS - farm.liquiditySlippage)) / BPS;
    @> minAmount1 = (minAmount1 * (BPS - farm.liquiditySlippage)) / BPS;
    // ...
}
```

In volatile market conditions or during large price movements, the actual amounts returned when removing liquidity can deviate by more than 30 percent from the TWAP-based estimate. When this happens, the transaction reverts because the minimum amount check fails.

Since `removeFarm()` relies on `_withdrawFromFarm()` , which calls `decreaseLiquidity()` , this slippage cap directly prevents the admin from removing a farm. Farm removal is also the only mechanism available to remove allocation points from a farm. If the removal is blocked, ETH continues to be distributed to a farm that the admin wants to shut down.

```
function removeFarm(bytes32 id, uint48 deadline)
    external
    beforeDeadline(deadline)
```



```
validFarmId(id)
nonReentrant
onlyOwner
{
    Farm storage farm = _farms.get(id);
    uint256 liquidityRemoved = farm.lp.liquidity;

    AccumulatedFees memory accumulatedFees;
    if (farm.lp.liquidity != 0) {
@>        (, accumulatedFees) = _withdrawFromFarm(farm, farm.lp.liquidity, deadline);
    }
    // ...
}
```

Recommendations

The recommendation is to allow a more generous slippage tolerance for liquidity removal operations, or to provide a separate configurable slippage parameter specifically for farm removal that allows a higher tolerance than the standard 30% cap used for regular liquidity operations.

[L-05] Swap event reports incorrect tax values when tax is disabled for `FarmKeeper`

Description

In `OlympusXHookLib::emitSwapEvent()`, the `taxBps` and `taxAmount` fields for sell swaps (OLY for ETH) are calculated based solely on whether `taxRouter` is a non-zero address. However, tax can be disabled for specific swaps such as those performed by `FarmKeeper` during rebalancing via `FarmKeeper::depositToFarm()`. In these cases, no tax is actually collected, but the event still reports non-zero tax values.

For exact output sells, `taxBps` is set to `currentTaxBps` whenever `taxRouter` is not equal to `address(0)`, and `taxAmount` is derived from it:

```
if (isExactOutput) {
@>    eventData.taxBps = taxRouter != address(0) ? currentTaxBps : 0;
@>    eventData.taxAmount = (eventData.amountOut * eventData.taxBps) / (BPS -
eventData.taxBps);
}
```

For exact input sells, `taxAmount` is taken from the `taxDelta` parameter which would be 0 when tax is disabled, but `taxBps` still reflects `currentTaxBps` if `taxRouter` is set, causing `eventData.amountOut` to be incorrectly reduced:

```
} else {
    uint256 grossAmountOut = uint256(int256(delta.amount0()));
    eventData.taxAmount = uint256(int256(taxDelta));
@>    eventData.taxBps = eventData.taxAmount > 0 ? currentTaxBps : 0;
    eventData.amountOut =
```



```
grossAmountOut > eventData.taxAmount ? grossAmountOut - eventData.taxAmount :  
grossAmountOut;  
}
```

In the exact input case, if `taxDelta` is 0 (no tax collected), `taxBps` correctly becomes 0. But in the exact output case, there is no such guard, so `taxBps` and `taxAmount` will be non-zero even when the swap was tax-exempt. This causes the `OlympusXSwap` event to report phantom tax amounts and an understated `amountOut` for tax-exempt exact output sells.

The recommendation is to pass the actual tax collection status (whether tax was applied for this specific swap) into `emitSwapEvent` rather than relying on whether `taxRouter` is a non-zero address, so that tax-exempt swaps correctly emit zero tax values.

[L-06] LiquidityAction event includes accrued fees in reported token amounts

Description

In `_afterAddLiquidity` and `_afterRemoveLiquidity`, the `delta` parameter passed to `emitLiquidityAction` is the combined `BalanceDelta` that includes both the principal liquidity change and any accrued fees. The second `BalanceDelta` parameter (fees accrued) is available in the function signature but is not used to subtract the fee component before emitting the event.

```
function _afterAddLiquidity(  
    address sender,  
    PoolKey calldata,  
    ModifyLiquidityParams calldata params,  
    @> BalanceDelta delta,  
    BalanceDelta,  
    bytes calldata  
) internal override returns (bytes4, BalanceDelta) {  
    @> OlympusXHookLib.emitLiquidityAction(true, params, sender, delta);  
    return (this.afterAddLiquidity.selector, delta);  
}  
  
function _afterRemoveLiquidity(  
    address sender,  
    PoolKey calldata,  
    ModifyLiquidityParams calldata params,  
    @> BalanceDelta delta,  
    BalanceDelta,  
    bytes calldata  
) internal override returns (bytes4, BalanceDelta) {  
    @> OlympusXHookLib.emitLiquidityAction(false, params, sender, delta);  
    return (this.afterRemoveLiquidity.selector, delta);  
}
```



As a result, the `amount0` and `amount1` values in the `LiquidityAction` event may be inflated because they include fees accrued on the position. Off-chain indexers or UIs consuming this event would display incorrect liquidity amounts that do not reflect the actual tokens added or removed.

The recommendation is to subtract the `feesAccrued` delta from the `delta` before passing it to `emitLiquidityAction`, or emit the fee amounts as separate fields in the event so consumers can distinguish between principal and fee components.

[L-07] Dust wstETH reward amount can revert harvest

Description

In `ValidatorVault::harvest()`, the `wstETHToStakers` amount is calculated as a percentage of `rewardWstETH`. This value is then passed directly to `StakingVault::distributeRewards()` without checking whether it is zero.

```
// In ValidatorVault::harvest()
uint256 wstETHToStakers = (rewardWstETH * STAKERS_BPS) / BPS;
// ...
IERC20(wstEth).approve(stakingVault, wstETHToStakers);
@> IStakingVault(stakingVault).distributeRewards(wstEth, wstETHToStakers);
```

However, `StakingVault::_distributeRewards()` includes a zero amount check that causes the call to revert when `_amount` is zero.

```
function _distributeRewards(address _token, uint256 _amount) internal returns (uint256
realAmount) {
@>     require(_amount > 0, StakingVault__ZeroAmount());
    // ...
}
```

When `rewardWstETH` is very small (for example, due to minimal staking yield over a short period), the integer division `(rewardWstETH * STAKERS_BPS) / BPS` can round down to zero. In this case, `harvest()` will always revert because `distributeRewards()` rejects zero amounts. This effectively blocks the entire harvest flow, preventing the buy and burn withdrawal and reinvestment steps from executing as well.

The recommendation is to only call `distributeRewards()` in `ValidatorVault::harvest()` when `wstETHToStakers` is greater than zero, allowing the rest of the harvest logic to proceed even when the staker portion rounds down to zero.



[L-08] GoodAccountingExecuted event excludes reward penalties from total

Description

In `StakingVault::goodAccounting()`, the `totalPenalties` counter only accumulates `principalPenalty` for each processed stake. The `rewardPenalty`, which represents penalties applied to inflation rewards, is not included even though it is calculated and applied during the same flow.

```
function goodAccounting(uint256[] calldata stakeIds)
    external
    nonReentrant
    returns (uint256 stakesProcessed, uint256 totalPenalties)
{
    // ...
    uint256 rewardPenalty;
    uint256 principalPenalty;
    (principalPenalty, rewardPenalty) =
    GracePeriodPenalties.calculateGracePeriodPenalties(
        stake.stakeInfo.maturityTs, stake.stakeInfo.olympusXAmount,
        accumulatedInflation, currentTimestamp
    );
    // ...

    // Update counters
    stakesProcessed++;
    totalPenalties = totalPenalties + principalPenalty;
}

// Emit summary event for good accounting batch
if (stakesProcessed > 0) {
    emit GoodAccountingExecuted(stakesProcessed, totalPenalties, currentDay);
}
```

As a result, the `GoodAccountingExecuted` event and the returned `totalPenalties` value understate the actual total penalties applied during the batch. Off-chain indexers and monitoring systems that rely on this event to track penalty amounts will report inaccurate figures.

The recommendation is to include `rewardPenalty` in the `totalPenalties` accumulator so that the emitted event and return value reflect the full penalty amount applied to each processed stake.



[L-09] StakeOwner mapping not cleared when stakes are removed

Description

In `StakingVault::_removeUserStake()`, when a stake is ended and removed from a user's active stake list, the `stakeOwner[_stakeId]` mapping is never cleared. The function only removes the stake ID from the `userStakeIds` array and deletes the `userStakeIndex` entry, but leaves `stakeOwner` pointing to the original owner.

```
function _removeUserStake(address _user, uint256 _stakeId) internal {
    uint256[] storage userStakes = userStakeIds[_user];
    uint256 length = userStakes.length;
    require(length > 0, StakingVault__StakeListEmpty());

    uint256 index = userStakeIndex[_user][_stakeId];

    if (index == length - 1) {
        userStakes.pop();
        delete userStakeIndex[_user][_stakeId];
    }
    return;

    uint256 lastStakeId = userStakes[length - 1];
    userStakes[index] = lastStakeId;
    userStakes.pop();

    userStakeIndex[_user][lastStakeId] = index;
    delete userStakeIndex[_user][_stakeId];
}
```

While `stakeOwner` is set during `_startStake()`, it is never deleted in `_removeUserStake()`. This means that after a stake is ended, `stakeOwner[stakeId]` still returns the original owner address instead of `address(0)`. Although `goodAccounting()` checks `stakeOwner` and skips stakes where the owner is `address(0)`, this stale mapping does not currently cause functional issues because the stake status is also checked. However, it results in unnecessary storage usage and can lead to incorrect results for external integrations or view functions that rely on `stakeOwner` to determine whether a stake is still active.

The recommendation is to add `delete stakeOwner[_stakeId]` inside `_removeUserStake()` to properly clean up the mapping when a stake is removed.



[L-10] Dust reward tokens trapped in staking vault due to payout pool zeroing

Description

In `StakingVault::_triggerCyclePayout()`, when distributing reward tokens to stakers, the function calculates `increment` as `(tokenReward * PRECISION) / _globalActiveShares` and then sets `tokenCyclePayoutPool[token][_cycleNo]` to zero. Due to integer division, a dust amount of reward tokens equal to `tokenReward - (increment * _globalActiveShares / PRECISION)` becomes unaccounted for after the pool is zeroed out. These tokens remain in the contract but are never distributed to any staker.

```
function _triggerCyclePayout(uint256 _cycleNo, uint256 _globalActiveShares, uint256
_currentContractDay) internal {
    if (_currentContractDay < nextCyclePayoutDay[_cycleNo]) return;

    nextCyclePayoutDay[_cycleNo] = ((_currentContractDay / _cycleNo) + 1) * _cycleNo;

    uint256 tokensLength = rewardTokens.length;

    if (_globalActiveShares > 0) {
        for (uint256 i = 0; i < tokensLength; i++) {
            address token = rewardTokens[i];
            uint256 tokenReward = tokenCyclePayoutPool[token][_cycleNo];

            if (tokenReward > 0) {
                uint256 increment = (tokenReward * PRECISION) / _globalActiveShares;

                if (increment > 0) {
                    cumulativeRewardPerShare[token][_cycleNo] += increment;
                    tokenCyclePayoutPool[token][_cycleNo] = 0;
                    // ...
                }
            }
        }
    }
    // ...
}
```

While the dust per individual payout is small, it accumulates over many cycles and across multiple reward tokens. Over the lifetime of the vault, these trapped amounts can become non-trivial, especially for high-value tokens with low decimal precision.

The recommendation is to roll over the dust remainder into the next cycle instead of zeroing out `tokenCyclePayoutPool[token][_cycleNo]`. This can be achieved by calculating the actual distributed amount and only subtracting that from the pool balance, leaving the remainder for the next payout trigger.

```
uint256 distributed = (increment * _globalActiveShares) / PRECISION;
tokenCyclePayoutPool[token][_cycleNo] = tokenReward - distributed;
```



[L-11] Unhandled chainlink revert bypasses twap fallback

Description

`_getPriceFromChainlink()` makes bare external calls to `priceFeed.latestRoundData()` and `priceFeed.decimals()` without `try/catch`. When Chainlink reverts (access control, deprecated feed), the revert propagates through `getLatestEthPriceUSD()`, completely bypassing the TWAP fallback. The fallback was designed for exactly this scenario but only handles stale/zero data (returns 0), not reverts.

```
// EthPrice.sol L104-126
function _getPriceFromChainlink(address priceFeedAddress, uint256 oracleStaleThreshold)
    internal view returns (uint256 price)
{
    AggregatorV3Interface priceFeed = AggregatorV3Interface(priceFeedAddress);
    (, int256 priceInt, , uint256 updatedAt, ) = priceFeed.latestRoundData(); // BARE CALL -
reverts propagate
    // ...
    uint8 decimals = priceFeed.decimals(); // BARE CALL - reverts propagate
}
```

PoC

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {EthPrice} from "@core/EthPrice.sol";

contract RevertingAggregator {
    bool public shouldRevert;

    function setRevert(bool _r) external {
        shouldRevert = _r;
    }

    function latestRoundData() external view returns (uint80, int256, uint256, uint256,
uint80) {
        require(!shouldRevert, "No access");
        return (1, 3000e8, 0, block.timestamp, 1);
    }

    function decimals() external pure returns (uint8) {
        return 8;
    }
}

contract M03_ChainlinkRevertTest is Test {
    RevertingAggregator feed;
    EthPrice ethPrice;

    function setUp() public {
        feed = new RevertingAggregator();
    }
}
```



```
ethPrice = new EthPrice(
    address(this),
    EthPrice.PriceConfig({
        chainlinkFeed: address(feed),
        oracleStaleThreshold: 3600,
        uniswapFallback: EthPrice.UniswapConfig({
            isUniV4: false,
            poolAddress: address(0),
            weth: address(0),
            poolData: bytes(""),
            twapPeriod: 0,
            usdTokenChainlinkFeed: address(0)
        })
    })
);

function test_M03_normalPriceWorks() public view {
    uint256 price = ethPrice.getLatestEthPriceUSD();
    assertEq(price, 3000e18);
}

function test_M03_revertBypassesTWAPFallback() public {
    feed.setRevert(true);
    vm.expectRevert("No access");
    ethPrice.getLatestEthPriceUSD();
}
```

Recommendations

Wrap `latestRoundData()` and `decimals()` calls in `try/catch`. On revert, return `0` to allow the existing TWAP fallback to activate.

[L-12] `setLaunchTime` can be called multiple times leading to time state corruption

Description

`StakingVault.setLaunchTime()` is a bare setter with `onlyAuthorized` (LaunchManager or `olympusX.owner()`). It has no re-invocation guard, no timestamp validation, and no check that the system is already live. A future timestamp causes `_updateCurrentDay()` to underflow and freeze all operations. A past timestamp jumps `currentDay` and corrupts inflation distribution.

```
// StakingVault.sol L521-524
function setLaunchTime(uint40 _launchTime) external onlyAuthorized {
    launchTime = _launchTime;
    emit LaunchTimeSet(_launchTime);
}
```



The `onlyAuthorized` modifier permits `olympusX.owner()` to call this directly, bypassing the single-call guard in `LaunchManager.startMintPhase()`.

PoC

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {Base} from "@test/Base.t.sol";

contract M03_SetLaunchTimeTest is Base {
    function test_M03_noReInvocationGuard() public {
        vm.startPrank(admin);
        stakingVault.setLaunchTime(uint40(block.timestamp));
        stakingVault.setLaunchTime(uint40(block.timestamp + 1));
        stakingVault.setLaunchTime(uint40(block.timestamp + 2));
        stakingVault.setLaunchTime(uint40(block.timestamp + 3));
        vm.stopPrank();
    }

    function test_M03_futureTimestamp_freezes() public {
        _simulateLaunch(true);
        vm.prank(admin);
        stakingVault.setLaunchTime(uint40(block.timestamp + 365 days));

        // Any call that triggers _updateCurrentDay() will underflow
        vm.expectRevert();
        stakingVault.claimAllRewards();
    }
}
```

Recommendations

Add a `launchTimeSet` boolean flag to prevent re-invocation. Validate `_launchTime <= uint40(block.timestamp)` to prevent future timestamps.

[L-13] `FarmKeeper_Removed` event misrepresents ETH/WETH pool data and omits revenue

Description

The `removeFarm` function in `FarmKeeper.sol` aggregates funds from three sources before distribution: the farm's idle principal (`currencyXBalance`), accumulated trading fees, and the `nativeBalance` (accumulated tax revenue). Furthermore, for WETH-based farms, the contract explicitly unwraps WETH into native ETH before transferring the total to the `TaxRouter`.



However, the `FarmKeeper_Removed` event only emits the `amount0` and `amount1` corresponding to the idle principal and the original token addresses (`curr0` , `curr1`). This results in two discrepancies:

1. Under-reporting: The event omits the value of accumulated fees and `nativeBalance` , causing the emitted values to be significantly lower than the actual assets distributed.
2. Type Mismatch: For WETH pools, the event implies WETH was distributed, while the contract actually distributed unwrapped native ETH.

Update the `FarmKeeper_Removed` event definition to include fields for `fees0` , `fees1` , and `nativeBalanceDistributed` , or emit the aggregated `ethAmount` calculated within the function to reflect the actual transfer value.

[L-14] Hardcoded Lido withdrawal limits may disrupt harvest after upgrade

Description

`ValidatorVault` hardcodes Lido's withdrawal request size bounds as constants:

```
uint256 public constant MIN_WITHDRAWAL_AMOUNT = 100; // 100 wei
uint256 MAX_WITHDRAWAL_AMOUNT = 1000 ether; // in _prepareWithdrawalAmounts()
```

These values are used in `_prepareWithdrawalAmounts()` to split `stETH` into valid withdrawal requests before calling `IWithdrawalQueue.requestWithdrawals()` . However, Lido's `WithdrawalQueueERC721` is deployed behind an upgradeable proxy, meaning a Lido DAO governance vote can replace the implementation with one that defines different `MIN_STETH_WITHDRAWAL_AMOUNT` and `MAX_STETH_WITHDRAWAL_AMOUNT` values. After such an upgrade, `ValidatorVault` 's hardcoded bounds would diverge from what the `WithdrawalQueue` actually enforces, causing `requestWithdrawals()` to revert with `RequestAmountTooSmall` or `RequestAmountTooLarge` on every `harvest()` call.

Recommendation

Consider making the minimum and maximum withdrawal amount bounds configurable via owner-restricted setter functions, rather than hardcoding them as constants.

[L-15] Missing initialization of `launchTime` and `lastInflationDistributionDay`

Description

When a new `stakingVault` is deployed and set in the `LaunchManager` contract, its `launchTime = 0` and `lastInflationDistributionDay = 0` . `LaunchManager.setStakingVault()` does not initialize either:



```
function setStakingVault(address _stakingVault) external override onlyOwner
notZeroAddress(_stakingVault) {
    stakingVault = StakingVault(payable(_stakingVault)); // ← no initialization
    emit StakingVaultSet(_stakingVault);
}
```

Since every time-dependent calculation in `StakingVault` uses `launchTime` as its epoch, all cycle payout schedules, inflation distribution timestamps, and stake maturity checks that derive from `currentDay` become completely incorrect:

```
// _updateCurrentDay():
uint40 newDay = (uint40(block.timestamp) - launchTime) / uint40(1 days) + 1;
// launchTime = 0 , currentDay inflated
```

With `lastInflationDistributionDay = 0`, the first call to `_distributeDailyInflation()` on the new vault attempts to distribute inflation for the entire period from `launchTime` to now.

Recommendation

Add a `setLastInflationDistributionDay()` function to `StakingVault` callable by the `LaunchManager`:

```
+function setLastInflationDay(uint40 _day) external onlyAuthorized {
+    lastInflationDistributionDay = _day;
+}
```

Then update `LaunchManager.setStakingVault()` to initialize the new vault correctly:

```
function setStakingVault(address _stakingVault) external override onlyOwner
notZeroAddress(_stakingVault) {
+    stakingVault.syncInflation();
+    StakingVault newVault = StakingVault(payable(_stakingVault));
+    newVault.setLaunchTime(launchTime);
+    newVault.setLastInflationDay(stakingVault.currentDay());
+    newVault.syncInflation();
    stakingVault = newVault;
    emit StakingVaultSet(_stakingVault);
}
```

[L-16] Missing validation for `PoolKey` allows unsupported pools to lock fund

Description

`place()`, `kill()`, and `_afterSwap()` accept any `PoolKey calldata key` without validating it matches `mainPoolKey`. The hook is designed exclusively for the ETH:OLY pool where `OLY` is a burnable token. If a Uniswap V4 pool with an unsupported token pair (e.g.



ETH:USDC) is deployed with `OlympusXHook` as its hook, users can deposit limit orders through `place()` successfully. However, once LP fees accumulate at any tick, all exit paths become permanently broken:

```
// handleLiquidityFees() – called in kill(), place(), _fillEpoch():
function handleLiquidityFees(
    IPoolManager poolManager,
    address validatorVault,
    address hookAddress,
    PoolKey calldata key,
    BalanceDelta deltaFee
) public {
    //...
    if (deltaFee.amount1() > 0) {
        uint128 amount1Fee = uint128(deltaFee.amount1());

        poolManager.mint(hookAddress, key.currency1.toId(), amount1Fee);
        poolManager.burn(hookAddress, key.currency1.toId(), amount1Fee);

        key.currency1.take(poolManager, hookAddress, amount1Fee, false);
        ERC20Burnable(Currency.unwrap(key.currency1)).burn(amount1Fee); //<--- will revert
        olyFees = amount1Fee;
    }

    //...
}
```

So when `kill()` reverts, the user cannot recover principal, and when `_fillEpoch()` reverts, the epoch never fills and as a result `withdraw()` always hits `NotFilled()`. The first fee accumulation at any tick permanently locks all depositor funds at that tick with no recovery path.

Recommendation

Add a pool key validation at the entry points of `place()`, `kill()`, and `_afterSwap()`:

```
require(key.toId() == mainPoolKey.toId(), "unsupported pool");
```

[L-17] V4 WETH pools route through intermediary instead of wrapping directly

Description

In `FarmKeeper`, the `isNativePool` check for V3 and V4 farms uses different logic. V3 farms check for WETH addresses via the `isWethToken` mapping, while V4 farms only check for native ETH (`address(0)`). This means a V4 farm with a WETH pair (e.g., WETH/USDC) is not considered a native pool and its ETH allocation is routed through an intermediary swap, incurring unnecessary fees and slippage.

For V3 farms, the check correctly identifies WETH pools as native:



```
// FarmKeeper.sol - enableV3Farm
@> bool isWethPool = (isWethToken[currency0] || isWethToken[currency1]);
// ...
_addFarm(id, params, poolKey, isWethPool, FarmType.V3);
```

For V4 farms, only `address(0)` is checked:

```
// FarmKeeper.sol - enableV4Farm
@> bool _isNativePool = (Currency.unwrap(params.poolKey.currency0) == address(0)
@> || Currency.unwrap(params.poolKey.currency1) == address(0));
// ...
_addFarm(idBytes, params, params.poolKey, _isNativePool, FarmType.V4);
```

When ETH is distributed to farms, pools where `isNativePool` is false accumulate ETH in `nativeBalance` instead of wrapping and crediting the WETH currency balance directly:

```
// FarmKeeperLib.sol - allocateEthToFarm
function allocateEthToFarm(...) external returns (uint256 wethToDeposit) {
@> if (!_isNativePool) {
unchecked {
@> farm.nativeBalance = farm.nativeBalance + amount;
}
return 0;
}

if (farmType == FarmType.V3) {
// Wraps ETH to WETH and credits directly to currency balance
address wethToken = isWethToken[currency0] ? currency0 : currency1;
IWETH9(wethToken).deposit{value: amount}();
// ...
}
}
```

On deposit, the accumulated `nativeBalance` for non-native V4 WETH pools is routed through `_swapV4EthForFarmToken`, which swaps ETH for one of the farm tokens via a configured intermediary pool:

```
function _swapV4EthForFarmToken(Farm storage farm, PoolData memory intermediaryPool,
uint48 deadline) private {
@> uint128 ethAmount = farm.nativeBalance > type(uint128).max ? type(uint128).max :
uint128(farm.nativeBalance);
if (ethAmount == 0) return;

@> SwapExactInResult memory result = _swapV4Currency(
SwapActions.SwapExactInParams({
amountIn: ethAmount, slippage: intermediaryPool.swapSlippage, twapPeriod:
intermediaryPool.twapPeriod
}),
intermediaryPool.poolKey,
Currency.wrap(address(0)),
intermediaryPool.poolKey.currency1,
deadline
);
// ...
}
```



V4 WETH pools with [good](#) liquidity exist on mainnet. For these pools, the contract could simply wrap ETH to WETH and credit the WETH currency balance directly. Instead, every ETH allocation cycle pays swap fees, slippage, and potential price impact through the intermediary pool.

The recommendation is to include mechanisms that could credit WETH currency balances directly for V4 WETH pools.

[L-18] Incomplete interfaces missing external functions from implementation

Description

Several interfaces in the protocol do not declare all external and public functions that exist in their corresponding implementations. While this does not cause runtime failures since most cross-contract calls either use concrete types or only call functions that are present in the interfaces, it creates an incomplete contract surface for integrators, off-chain tooling, and future contracts that may depend on these interfaces.

The following functions are missing from each interface:

`ILaunchManager` is missing:

```
function extendMintPhase() external;
function setTaxRouter(address _taxRouter) external;
function setOlympusXHook(address payable _olympusXHook) external;
function getVestingSchedules() external view returns (VestingSchedule[] memory);
function calculateSqrtPriceX96(uint256 _amount0, uint256 _amount1) public pure returns (uint160);
```

`IStakingVault` is missing:

```
function setLaunchTime(uint40 _launchTime) external;
function numCheckpoints(address account) public view returns (uint32);
function checkpoints(address account, uint32 pos) public view returns (Checkpoints.Checkpoint208 memory);
function totalSupply() public view returns (uint256);
```

`IFarmKeeper` is missing:

```
function setTaxRouter(address _taxRouter) external;
function setOlympusXHook(address _olympusXHook) external;
function farmTypes(bytes32 id) external view returns (FarmType);
function isNativePool(bytes32 id) public view returns (bool);
function lastHarvestTime(bytes32 id) public view returns (uint48);
function getIntermediaryPool(bytes32 id) external view returns (PoolData memory);
```

`IOlympusXAuction` is missing all public state variable getters including `currentCycle`, `nextCycleTimestamp`, `auctionStartTime`, `auctionEndTime`, `totalEthDeposited`, `olympusXDistributedPerCycle`, `totalClaimedPerCycle`, and `totalReservedTokens`.



The recommendation is to add the missing function declarations to each interface and correct the inaccurate natspec descriptions to ensure the interfaces fully represent their implementations.

[L-19] `harvestFarm` event emits current fee instead of total distributed fees

Description

`FarmKeeper_Harvested` event emits `params.fee0` and `params.fee1` (the fees from the current `_collectFees()` call) instead of `accumulatedFees.accumulatedFeeCurrency0/1` (the total amount actually distributed). If `collectFees(farmId)` is called independently before `harvestFarm(farmId)`, the accumulated fees include the prior collection, but the event only reports the harvest-time collection. Off-chain analytics and dashboards consuming this event will underreport the actual distribution.

Recommendation: Emit `accumulatedFees.accumulatedFeeCurrency0` and `accumulatedFees.accumulatedFeeCurrency1` in the event instead of `params.fee0` / `params.fee1`.

[L-20] `ClaimRewards` skips `distributeDailyInflation` unlike all other entry points

Description

`_claimRewardForToken()` (called by `claimRewards()`) calls `_updateCurrentDay()` and `_triggerPayouts()` but does not call `_distributeDailyInflation()`. In contrast, `claimAllRewards()` calls all three:

```
// claimAllRewards (L328-331)
_updateCurrentDay();
_distributeDailyInflation(); // present
_triggerPayouts();

// _claimRewardForToken (L931-933)
_updateCurrentDay();
// missing: _distributeDailyInflation()
_triggerPayouts();
```

When a user calls `claimRewards()`, `lastInflationDistributionDay` is not advanced and pending daily inflation is not credited to the accumulated reward pool. The user receives rewards based on stale `_accRewardPerShare` values. The inflation is eventually distributed when another function calls `_distributeDailyInflation()` (e.g., `startStake`, `stakeEnd`, `claimAllRewards`), so no tokens are permanently lost, but users calling `claimRewards` receive fewer rewards than expected at that moment.



Add `_distributeDailyInflation()` to `_claimRewardForToken()` so reward state is fully current regardless of which claim path users take.

[L-21] `MarketCapLib.getMarketCapTier` has wrong tier boundary comment

Description

Every inline comment inside `getMarketCapTier()` states incorrect market cap boundaries. The constants are correct (\$25M / \$100M / \$1B), but the comments describe a completely different fee schedule (\$100M / \$250M / \$500M) that does not exist anywhere in the codebase or whitepaper.

```
// MarketCapLib.sol L157-166
function getMarketCapTier(uint256 marketCap) public pure returns (uint256 taxBps) {
    if (marketCap < TIER1_MAX) { // TIER1_MAX = $25M
        return TIER1_TAX_BPS; // 38% for <$100M ← WRONG: should be <$25M
    } else if (marketCap < TIER2_MAX) { // TIER2_MAX = $100M
        return TIER2_TAX_BPS; // 28% for $100M-$250M ← WRONG: should be $25M-$100M
    } else if (marketCap < TIER3_MAX) { // TIER3_MAX = $1B
        return TIER3_TAX_BPS; // 18% for $250M-$500M ← WRONG: should be $100M-$1B
    } else {
        return TIER4_TAX_BPS; // 8% for >$500M ← WRONG: should be >$1B
    }
}
```

The constants (defined L27-29) are:

- `TIER1_MAX = 25_000_000e18` (\$25M)
- `TIER2_MAX = 100_000_000e18` (\$100M)
- `TIER3_MAX = 1_000_000_000e18` (\$1B)

The comments reference a ghost schedule that does not match any tier boundaries. This creates a documentation trap for developers, integrators, and governance.

Correct all four comments to match the actual constants:

```
- return TIER1_TAX_BPS; // 38% for <$100M
+ return TIER1_TAX_BPS; // 38% for <$25M market cap

- return TIER2_TAX_BPS; // 28% for $100M-$250M
+ return TIER2_TAX_BPS; // 28% for $25M-$100M market cap

- return TIER3_TAX_BPS; // 18% for $250M-$500M
+ return TIER3_TAX_BPS; // 18% for $100M-$1B market cap

- return TIER4_TAX_BPS; // 8% for >$500M
+ return TIER4_TAX_BPS; // 8% for >$1B market cap
```



[L-22] Claim all buyAndBurnWithdrawals DoS when any request is unfinalized

Description

`ValidatorVault.claimAllBuyAndBurnWithdrawals()` passes the entire `pendingBuyAndBurnWithdrawals` array to Lido's `claimWithdrawalsTo()`, which reverts if any request in the batch is not yet finalized. When multiple harvest cycles queue withdrawal requests that finalize at different times, the batch function becomes unusable and finalized ETH is stuck until all requests finalize. An individual `claimBuyAndBurnWithdrawal(requestId)` alternative exists but requires knowing specific request IDs. Filter for finalized-only requests in the batch function by querying `getWithdrawalStatus` and only claiming requests where `isFinalized` is equal to true.

[L-23] Bonus formula deviates from whitepaper giving users fewer shares

Description

`StakingCalculations.calculateShares()` computes the time bonus as `(stakeDays - 88) / 1688 * 40000` instead of the whitepaper's `stakeDays / 1776 * 40000`:

```
// StakingCalculations.sol L30-33
bonusBPS = Math.mulDiv(
    MAX_BONUS_BPS,
    stakeDays - MIN_STAKE_DAYS, // shifted by -88
    MAX_STAKE_DAYS - MIN_STAKE_DAYS, // 1688 instead of 1776
    Math.Rounding.Floor
);
```

By subtracting `MIN_STAKE_DAYS` from both the numerator and denominator, the formula maps `[88, 1776]` to `[0, 40000]` BPS instead of `[0, 1776]` to `[0, 40000]` BPS. This systematically underpays every staker who does not stake for the full 1,776 days.

At minimum duration (88 days), users receive zero bonus when the whitepaper promises 1,981 BPS (~19.81%). At 365 days, the code gives 6,563 BPS versus the whitepaper's 8,220 BPS -- a 20.2% reduction. The two formulas converge only at maximum duration (1,776 days = 40,000 BPS).

For a 1,000,000 OLY stake at 88 days, the code awards zero bonus shares while the whitepaper formula would award an additional approximately 19.81% bonus shares. This compounds across all stakers, diluting every non-max-duration participant's share of inflation rewards.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;
```



```
import {Test} from "forge-std/Test.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
import {StakingCalculations} from "@libs/StakingCalculations.sol";

contract M08_BonusFormulaDeviationTest is Test {
    uint256 constant PRECISION = 1e24;
    uint16 constant MIN_STAKE_DAYS = 88;
    uint16 constant MAX_STAKE_DAYS = 1776;
    uint16 constant MAX_BONUS_BPS = 400_00;
    uint16 constant BPS = 100_00;
    uint256 constant START_SHARE_RATE = 800 ether;

    function _whitepaperBonusBPS(uint16 stakeDays) internal pure returns (uint256) {
        return Math.mulDiv(MAX_BONUS_BPS, stakeDays, MAX_STAKE_DAYS, Math.Rounding.Floor);
    }

    function _codeBonusBPS(uint16 stakeDays) internal pure returns (uint256) {
        return Math.mulDiv(MAX_BONUS_BPS, stakeDays - MIN_STAKE_DAYS, MAX_STAKE_DAYS -
MIN_STAKE_DAYS, Math.Rounding.Floor);
    }

    function test_M08_zeroBonus_atMinimumStake() public pure {
        uint256 codeBPS = _codeBonusBPS(88);
        uint256 wpBPS = _whitepaperBonusBPS(88);
        assertEq(codeBPS, 0);
        assertEq(wpBPS, 1981);
    }

    function test_M08_underpayment_acrossAllDurations() public pure {
        uint16[4] memory days_ = [uint16(88), uint16(365), uint16(888), uint16(1776)];
        for (uint256 i = 0; i < days_.length; i++) {
            uint256 wp = _whitepaperBonusBPS(days_[i]);
            uint256 code = _codeBonusBPS(days_[i]);
            if (days_[i] == MAX_STAKE_DAYS) {
                assertEq(wp, code);
            } else {
                assertGt(wp, code);
            }
        }
    }

    function test_M08_shareShortfall_1M_OLY_88days() public pure {
        uint256 amount = 1_000_000 ether;
        uint256 baseShares = (amount * PRECISION) / START_SHARE_RATE;
        uint256 wpTotal = baseShares + (baseShares * _whitepaperBonusBPS(88)) / BPS;
        uint256 codeTotal = baseShares + (baseShares * _codeBonusBPS(88)) / BPS;
        assertGt(wpTotal, codeTotal);
        uint256 actual = StakingCalculations.calculateShares(amount, 88, START_SHARE_RATE);
        assertEq(actual, codeTotal);
    }

    function test_M08_365day_deviation() public pure {
        uint256 wpBPS = _whitepaperBonusBPS(365);
        uint256 codeBPS = _codeBonusBPS(365);
        assertEq(wpBPS, 8220);
        assertEq(codeBPS, 6563);
    }
}
```



```

        assertGt(wpBPS, codeBPS);
    }
}

```

```

[PASS] test_M08_365day_deviation() (gas: 1870)
[PASS] test_M08_shareShortfall_1M_OLY_88days() (gas: 3672)
[PASS] test_M08_underpayment_acrossAllDurations() (gas: 7177)
[PASS] test_M08_zeroBonus_atMinimumStake() (gas: 1768)
Suite result: ok. 4 passed; 0 failed; 0 skipped

```

Recommendations

Replace the shifted formula with a linear interpolation from 0 to `MAX_BONUS_BPS` across the full `[0, MAX_STAKE_DAYS]` range to match the white paper:

```

bonusBPS = Math.mulDiv(
    MAX_BONUS_BPS,
    - stakeDays - MIN_STAKE_DAYS,
    - MAX_STAKE_DAYS - MIN_STAKE_DAYS,
    + stakeDays,
    + MAX_STAKE_DAYS,
    Math.Rounding.Floor
);

```

If the current behavior is intentional, update the whitepaper to document the shifted formula.

[L-24] `addLiquidityTokens` missing `nonReentrant` - ERC-777 balance inflation

Description

`FarmKeeper.addLiquidityTokens()` is the **only** state-modifying public function without the `nonReentrant` modifier. It also has **no access control**. The balance-before/after pattern in `FarmKeeperLib.addLiquidityTokens()` is not safe under reentrancy — an ERC-777 `tokensReceived` callback can re-enter, causing the outer call's delta to double count the inner transfer.

```

// FarmKeeper.sol L392 — NO nonReentrant, NO access control
function addLiquidityTokens(bytes32 id, uint256 amount0, uint256 amount1) public payable
validFarmId(id) {
    Farm storage farm = _farms.get(id);
    FarmKeeperLib.addLiquidityTokens(farm, amount0, amount1, msg.value);
}

```

Under reentrancy, transferring `X` tokens twice (outer + inner) records `3X` in `farm.currency1Balance` instead of `2X` — a 50% inflation.

PoC:



```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Base} from "@test/Base.t.sol";
import {FarmKeeper} from "@core/FarmKeeper.sol";

/// @title M05_AddLiquidityReentrancyTest
/// @notice Demonstrates that FarmKeeper.addLiquidityTokens() (L392) lacks the
///         `nonReentrant` modifier present on all other state-modifying functions.
///         Under ERC-777 reentrancy, the balance-before/after delta pattern inflates
///         recorded liquidity by double-counting the outer call's contribution.
contract M05_AddLiquidityReentrancyTest is Base {
    function test_M05_addLiquidityTokensCallable() public view {
        // Confirm addLiquidityTokens exists on FarmKeeper (inherits SwapActions)
        // The function signature: addLiquidityTokens(bytes32 id, uint256 amount0, uint256
amount1)
        // It is `public payable validFarmId(id)` – NO nonReentrant modifier
        // Compare with depositToFarm, collectFees, harvestFarm – all have nonReentrant
        bytes4 selector = farmKeeper.addLiquidityTokens.selector;
        assertTrue(selector != bytes4(0));
    }

    function test_M05_balanceBeforeAfterBrokenUnderReentrancy() public {
        uint256 contractBalance = 0;
        uint256 X = 100;
        uint256 actualTransferred;
        uint256 recordedBalance;

        // Outer call: balanceBefore = 0
        uint256 outerBalanceBefore = contractBalance;
        contractBalance += X;
        actualTransferred += X;

        // ERC-777 callback – inner call re-enters
        {
            uint256 innerBalanceBefore = contractBalance; // = X
            contractBalance += X; // inner transfer
            actualTransferred += X;
            uint256 innerActual = contractBalance - innerBalanceBefore; // = X (correct)
            recordedBalance += innerActual; // recorded: X
        }

        // Outer resumes: delta = currentBalance - outerBalanceBefore = 2X - 0 = 2X
(INFLATED)
        uint256 outerActual = contractBalance - outerBalanceBefore;
        recordedBalance += outerActual; // recorded: X + 2X = 3X

        assertEquals(actualTransferred, 2 * X);
        assertEquals(recordedBalance, 3 * X); // inflated by X
    }
}
```

Recommendations

Add `nonReentrant` modifier to `addLiquidityTokens()`. Consider also adding access control (`onlyOwner`).



[L-25] `addRewardToken` has no cap unbounded gas DoS on core operation

Description

`StakingVault.addRewardToken()` has no `MAX_REWARD_TOKENS` limit and no `removeRewardToken()` function. Every core operation (`claimAllRewards`, `endStake`, `distributeRewards`, `goodAccounting`) iterates over the full `rewardTokens` array multiplied by 5 payout cycles. Adding enough tokens causes gas costs to exceed block limits, permanently DoS-ing the staking system.

The whitepaper describes a `removeRewardToken()` function that would allow cleanup of unwanted tokens, but this function does not exist in the implementation. Its absence means once a token is added it is permanent, making the DoS vector irreversible without a contract upgrade.

```
// StakingVault.sol L484-492
function addRewardToken(address _token) external onlyAuthorized {
    if (isRewardToken[_token]) revert StakingVault__TokenAlreadyAdded();
    uint256 index = rewardTokens.length;
    rewardTokens.push(_token);
    isRewardToken[_token] = true;
    emit RewardTokenAdded(_token, index);
}
```

No cap, no removal mechanism. Once added, tokens are permanent.

PoC:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import {Base} from "@test/Base.t.sol";

contract M04_UnboundedRewardTokensTest is Base {
    function test_M04_noCapOnRewardTokens() public {
        vm.startPrank(admin);
        for (uint256 i = 1; i <= 50; i++) {
            stakingVault.addRewardToken(address(uint160(i)));
        }
        vm.stopPrank();
    }

    function test_M04_noRemoveFunction() public {
        vm.startPrank(admin);
        stakingVault.addRewardToken(address(uint160(1)));
        // No removeRewardToken() exists – once added, permanent
        // All core operations iterate: rewardTokens.length x 5 cycles
        vm.stopPrank();
    }
}
```



Recommendations

Add a `MAX_REWARD_TOKENS` constant (e.g., 10) enforced in `addRewardToken()`, and implement a `removeRewardToken()` function for cleanup.